2. Single Responsibility Principle (SRP) in Java

2.1 Introduction

The **Single Responsibility Principle (SRP)** is the first and most foundational of the **SOLID** principles. It states that a class should have **only one reason to change**, meaning it should have only one job or responsibility.

2.2 Problem It Solves

When classes are responsible for too many things, they become:

- Hard to test
- Hard to modify
- Hard to understand
- Prone to bugs and regressions

2.3 Definition of SRP

"A class should have only one reason to change."
This means that each class should be focused on a single task or responsibility.

2.4 Real-World Analogy

A chef should cook, not serve customers and do the billing. If the chef's role changes due to billing rules or service changes, it creates chaos.

2.5 Symptoms of SRP Violation

- One class handles UI, database, and business logic
- Too many methods doing unrelated things
- Large number of code changes ripple through multiple parts

2.6 Benefits of SRP

- Better maintainability
- Improved testability
- Loose coupling and high cohesion
- Enhanced reusability

2.7 Implementation Guidelines

- Identify all responsibilities a class has
- Break them into cohesive components
- Use interfaces or services to delegate work

2.8 Examples

Example1: Violation: Book Printer (SRP Violation vs SRP Adherence)

```
package com.mannemlokesh.violation.example1;

//SRP Violation
```

```
public class Book {
	String title, author;

	public Book(String title, String author) {
		this.title = title;
		this.author = author;
	}

	public void print() {
		System.out.println("Book: " + title + " by " + author);
	}
}
```

Example1: Compliant : Book Printer (SRP Violation vs SRP Adherence)

```
package com.mannemlokesh.compliant.example1;

//SRP Compliant
class Book {
	String title, author;

	public Book(String title, String author) {
		this.title = title;
		this.author = author;
	}
}

class BookPrinter {
	public void print(Book book) {
		System.out.println("Book: " + book.title + " by " + book.author);
	}
}

public class BookDemo {
	public static void main(String[] args) {
		Book book = new Book("Clean Code", "Robert Martin");
		BookPrinter printer = new BookPrinter();
		printer.print(book);
	}
}
```

Output:

```
Book: Clean Code by Robert Martin
```

Example2: Violation: Invoice Management

```
package com.mannemlokesh.violation.example2;

//SRP Violation: One class does too much
public class Invoice {
	double amount;

	public Invoice(double amount) {
		this.amount = amount;
	}
```

```
	public void calculateTax() {
		System.out.println("Tax: " + amount * 0.18);
	}

	public void printInvoice() {
		System.out.println("Invoice: " + amount);
	}

	public void saveToDatabase() {
		System.out.println("Saving invoice to DB...");
	}
}
```

Example2: Compliant : Invoice Management

```
package com.mannemlokesh.compliant.example2;

//SRP Compliant Version
class Invoice {
	double amount;

	public Invoice(double amount) {
		this.amount = amount;
	}
}

class TaxCalculator {
	public double calculateTax(Invoice invoice) {
		return invoice.amount * 0.18;
	}
}

class InvoicePrinter {
	public void print(Invoice invoice) {
		System.out.println("Invoice: " + invoice.amount);
	}
}

class InvoiceRepository {
	public void save(Invoice invoice) {
		System.out.println("Saving invoice to DB...");
	}
}

public class InvoiceClient {
	public static void main(String[] args) {
		Invoice invoice = new Invoice(1000);
		new InvoicePrinter().print(invoice);
		double tax = new TaxCalculator().calculateTax(invoice);
		System.out.println("Tax: " + tax);
		new InvoiceRepository().save(invoice);
	}
}
```

Output:

```
Invoice: 1000.0
Tax: 180.0
Saving invoice to DB...
```

Example3: Complaint : Invoice Management

```
package com.mannemlokesh.compliant.example3;

//SRP: Each class has one reason to change
class User {
	String username;

	public User(String username) {
		this.username = username;
	}
}

class AuthService {
	public boolean authenticate(User user) {
		return user.username != null && !user.username.isEmpty();
	}
}

class AuditLogger {
	public void log(String message) {
		System.out.println("[Audit] " + message);
	}
}

public class UserClient {
	public static void main(String[] args) {
		User user = new User("Lokesh");
		AuthService auth = new AuthService();
		AuditLogger logger = new AuditLogger();

		if (auth.authenticate(user)) {
			logger.log("User authenticated: " + user.username);
		} else {
			logger.log("Authentication failed.");
		}
	}
}
```

Output:

```
[Audit] User authenticated: Lokesh
```

2.9 Common Mistakes

- Over-separating responsibilities (too granular)
- Grouping responsibilities based on data instead of behavior
- Thinking SRP only applies to classes (applies to methods and modules too)

2.10 SRP vs Separation of Concerns

Principle	Focus
SRP	Class should have one reason to change
Separation of Concerns	Divide system into distinct sections

2.11 When to Apply

- When your class grows in size and responsibility
- When unit testing becomes hard
- When a class changes for multiple unrelated reasons

2.12 Summary

Feature	Description
Intent	One class = one responsibility
Use Case	Logging, business logic, data persistence
Key Benefit	Modular, testable, maintainable code

The Single Responsibility Principle is a guiding light for clean, focused, and resilient design. Mastering SRP leads to software that is easier to grow, test, and evolve.